Northstar

AWS Three-Tier Architecture and Resilience Lab

Arav Sharma | Built October 2026 in AWS eu-west-2 (London)

Northstar is a small web application that I deployed on AWS using a standard three-tier layout: a public load balancer, a private application server and a private database. I built it by hand in the AWS console to understand how each piece fits together, what each security control actually does, and how to find the problem when something stops working. After getting it running end to end, I broke it on purpose twice and recorded how the failure showed up and how I fixed it.

This document walks through the design, the decisions behind it, the evidence that it worked, the two failure tests, and what I would change before running something like this in production.

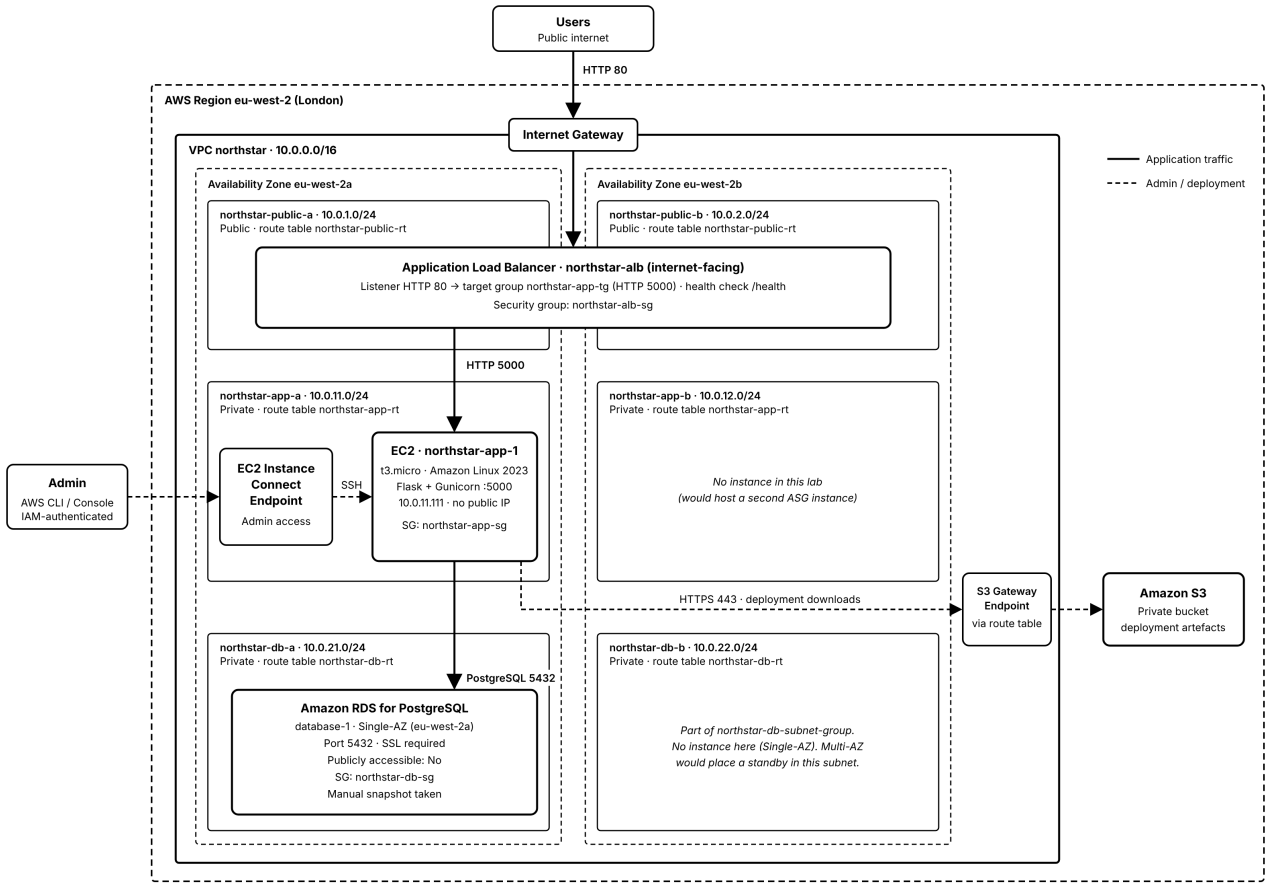


Figure 1. Architecture. Solid arrows show user traffic; dashed arrows show admin and deployment traffic.

1. Overview

How a request flows

A user sends an HTTP request to the load balancer's public DNS name. The request enters the VPC through the Internet Gateway and reaches the Application Load Balancer (ALB), which sits in the two public subnets. The ALB forwards it on port 5000 to the EC2 instance in a private subnet, where Gunicorn runs the Flask application. If the request needs data, the application opens an SSL connection on port 5432 to the PostgreSQL database on Amazon RDS, which sits in its own private subnets. The response returns along the same path.

Neither the EC2 instance nor the database has a public IP address, so the only way to reach the application from the internet is through the load balancer.

Components

Component	What I used	Why
Network	Custom VPC `10.0.0.0/16` with six /24 subnets: public, app and database, each in two Availability Zones	Separates each tier so that routing and security rules can be different for each one
Load balancer	Internet-facing Application Load Balancer, HTTP listener on port 80, health check on `/health`	Gives the app a public entry point without exposing the server, and stops sending traffic to an unhealthy server
Compute	One t3.micro EC2 instance, Amazon Linux 2023, private IP `10.0.11.111`, IMDSv2 required	Small and cheap for a lab; IMDSv2 protects instance credentials from a common class of SSRF attacks
Application	Python Flask app served by Gunicorn on port 5000	Gunicorn is a production WSGI server; the Flask built-in server is only meant for development
Database	Amazon RDS for PostgreSQL, Single-AZ in eu-west-2a, not publicly accessible, SSL required	Managed backups, patching and snapshots, kept off the internet entirely
Admin access	EC2 Instance Connect Endpoint	SSH into a private instance without a bastion host or a public SSH port
Deployment	Private S3 bucket, reached through an S3 Gateway VPC Endpoint	The private instance can download application files without any internet access or NAT gateway
Security	Three security groups that reference each other: ALB, app and database	Each tier only accepts traffic from the tier directly in front of it

Scope and cost

To keep costs low I ran a single EC2 instance and a Single-AZ database, used HTTP rather than HTTPS, and did not use a NAT gateway. The subnets were still built across two Availability Zones, because the ALB needs subnets in at least two zones and an RDS DB subnet group must also cover two zones. That also means the network is already laid out for a highly available version. I tore everything down after testing. Section 10 lists the changes I would make for production.

2. Network

VPC and subnets

I created a VPC with the range `10.0.0.0/16` and split it into six /24 subnets (256 addresses each). The third octet tells you the tier: 1 and 2 are public, 11 and 12 are the application tier, and 21 and 22 are the database tier. The "a" subnets are in eu-west-2a and the "b" subnets in eu-west-2b. Using a predictable numbering scheme made it much easier to read route tables and security group rules later, because an address immediately tells you which tier it belongs to.

Subnet	CIDR	Availability Zone	Tier	Route table
northstar-public-a	10.0.1.0/24	eu-west-2a	Public	northstar-public-rt
northstar-public-b	10.0.2.0/24	eu-west-2b	Public	northstar-public-rt
northstar-app-a	10.0.11.0/24	eu-west-2a	Application	northstar-app-rt
northstar-app-b	10.0.12.0/24	eu-west-2b	Application	northstar-app-rt
northstar-db-a	10.0.21.0/24	eu-west-2a	Database	northstar-db-rt
northstar-db-b	10.0.22.0/24	eu-west-2b	Database	northstar-db-rt

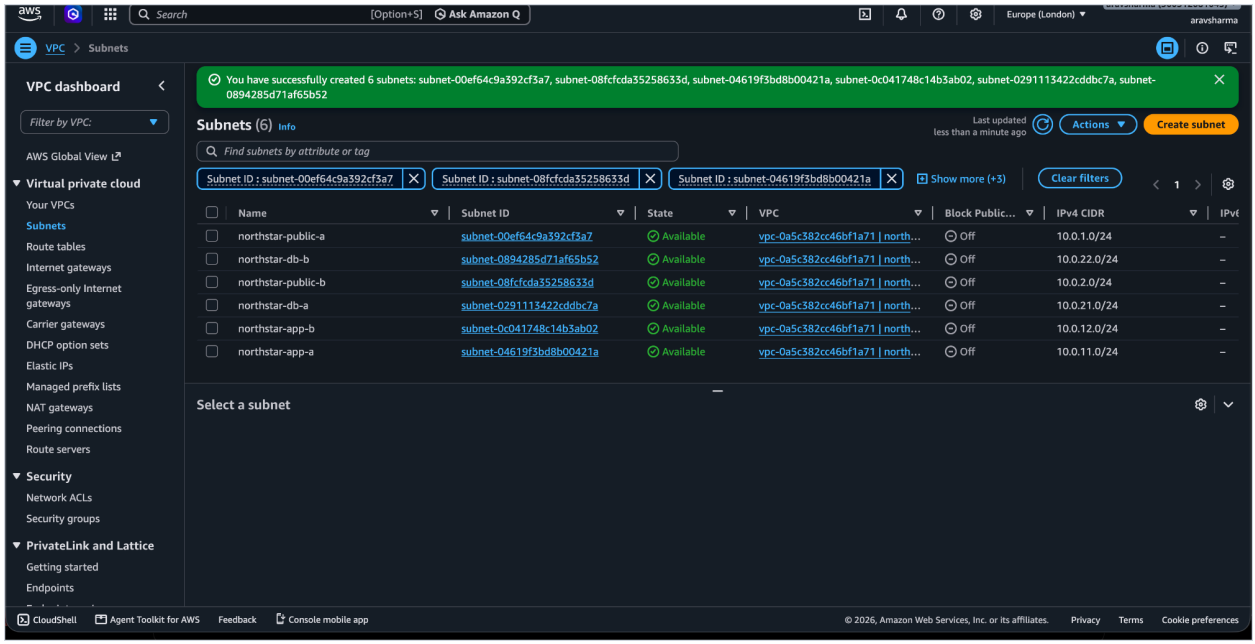


Figure 2. The six subnets after creation, all in northstar-vpc.

Route tables

A subnet is only "public" because its route table has a route to an Internet Gateway. I created one route table per tier and explicitly associated each subnet with the right one, rather than relying on the VPC's default main route table:

- `northstar-public-rt` has the local VPC route plus `0.0.0.0/0` to the Internet Gateway. Only the two public subnets use it, and the only resource in those subnets is the ALB.
- `northstar-app-rt` has the local route plus a route to the S3 prefix list, which AWS adds automatically when the S3 Gateway Endpoint is attached to it (section 4). There is no default route, so the app servers cannot reach the internet.
- `northstar-db-rt` has only the local route. The database can talk to other addresses inside the VPC and nothing else.

Because none of the private route tables has a route to the internet, even a misconfigured security group could not expose the application or database directly.

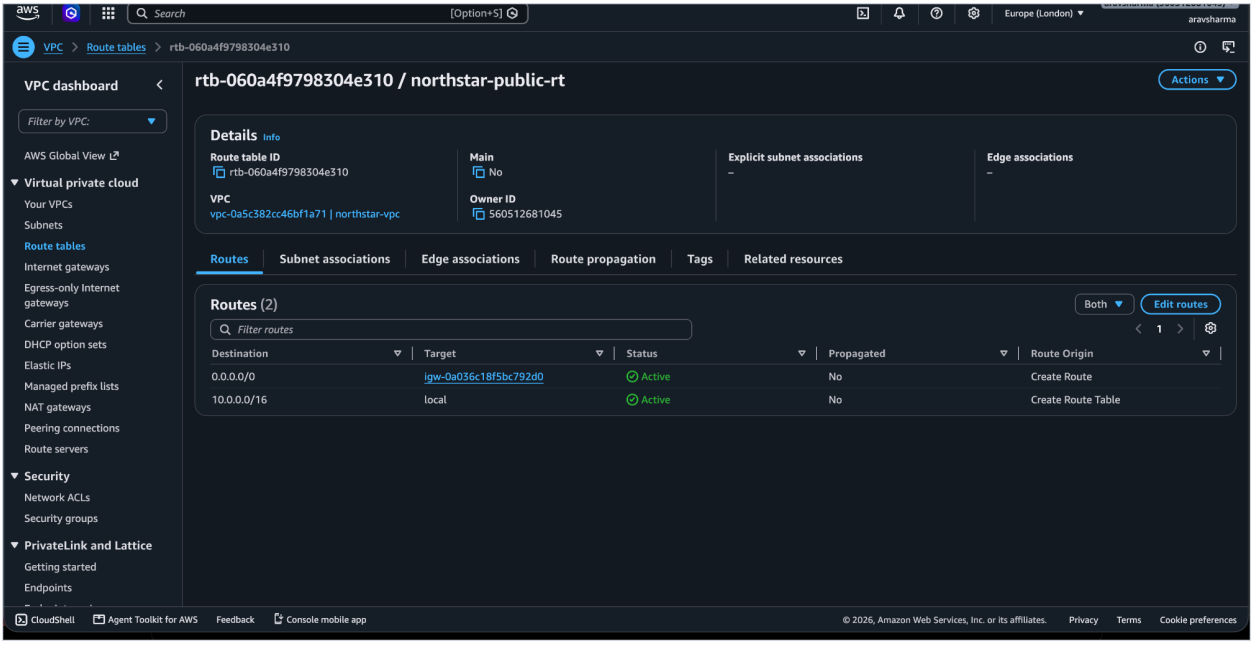


Figure 3. northstar-public-rt: the 0.0.0.0/0 route to the Internet Gateway is what makes these subnets public.

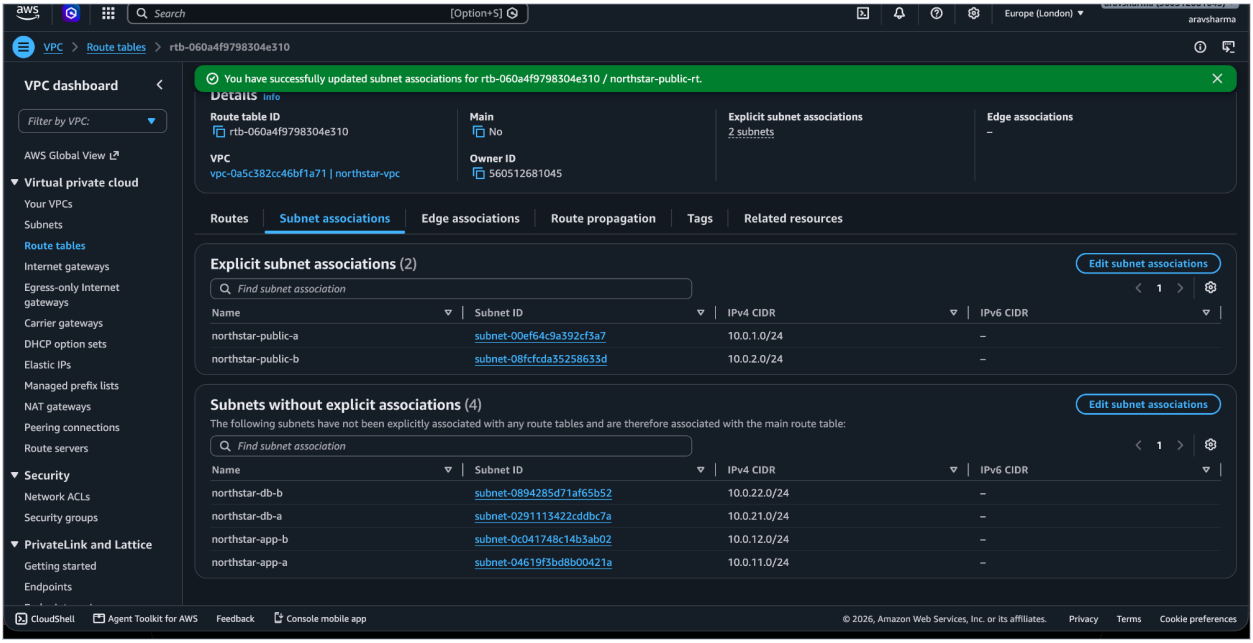


Figure 4. Only northstar-public-a and northstar-public-b are associated with the public route table.

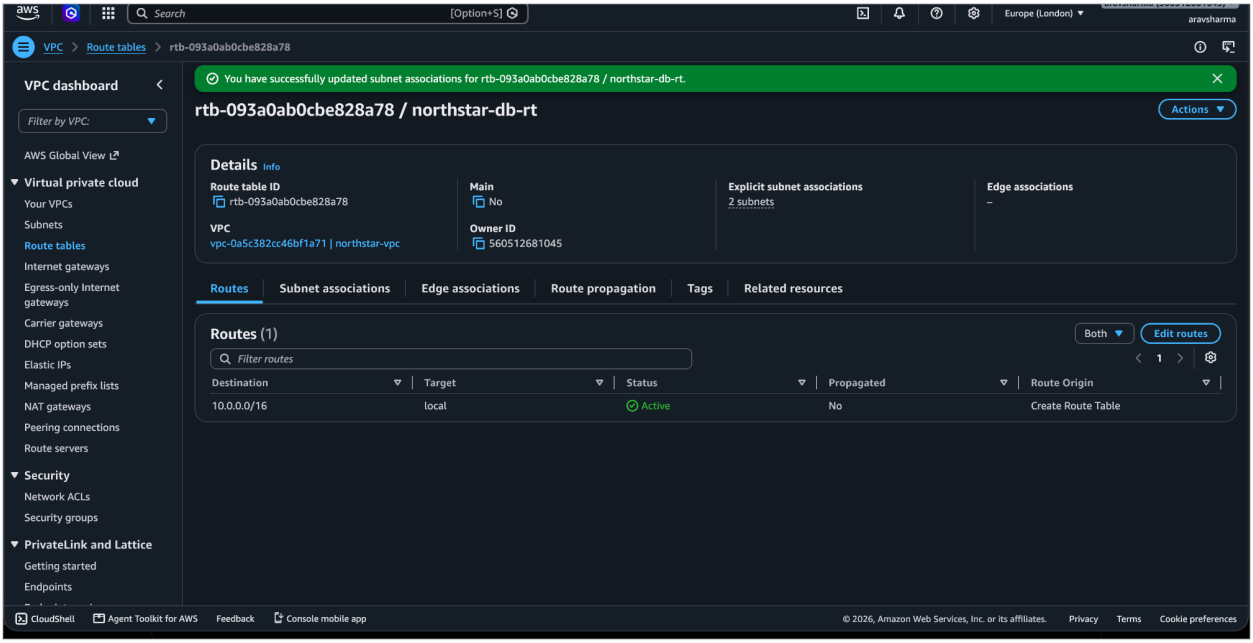


Figure 5. northstar-db-rt contains only the local 10.0.0.0/16 route, so the database subnets have no path to the internet.

3. Security groups

Security groups are stateful firewalls attached to each resource. Instead of allowing traffic from IP ranges, the app and database groups allow traffic from another security group. A rule such as “allow port 5432 from northstar-app-sg” means “allow it from anything that has northstar-app-sg attached”. That keeps working if the instance is replaced or its IP changes, and it would automatically cover extra instances in an Auto Scaling Group.

Security group	Inbound port	Allowed source	Purpose
northstar-alb-sg	HTTP 80	0.0.0.0/0	Public web traffic to the load balancer
northstar-app-sg	TCP 5000	northstar-alb-sg	Only the ALB can reach the application
northstar-app-sg	SSH 22	EC2 Instance Connect Endpoint	Admin shell access from inside the VPC
northstar-db-sg	PostgreSQL 5432	northstar-app-sg	Only the application can reach the database

The result is a chain: the internet can reach the ALB, the ALB can reach the app, and the app can reach the database. Nothing can skip a step. For example, there is no rule that would let the ALB or the internet talk to the database directly.

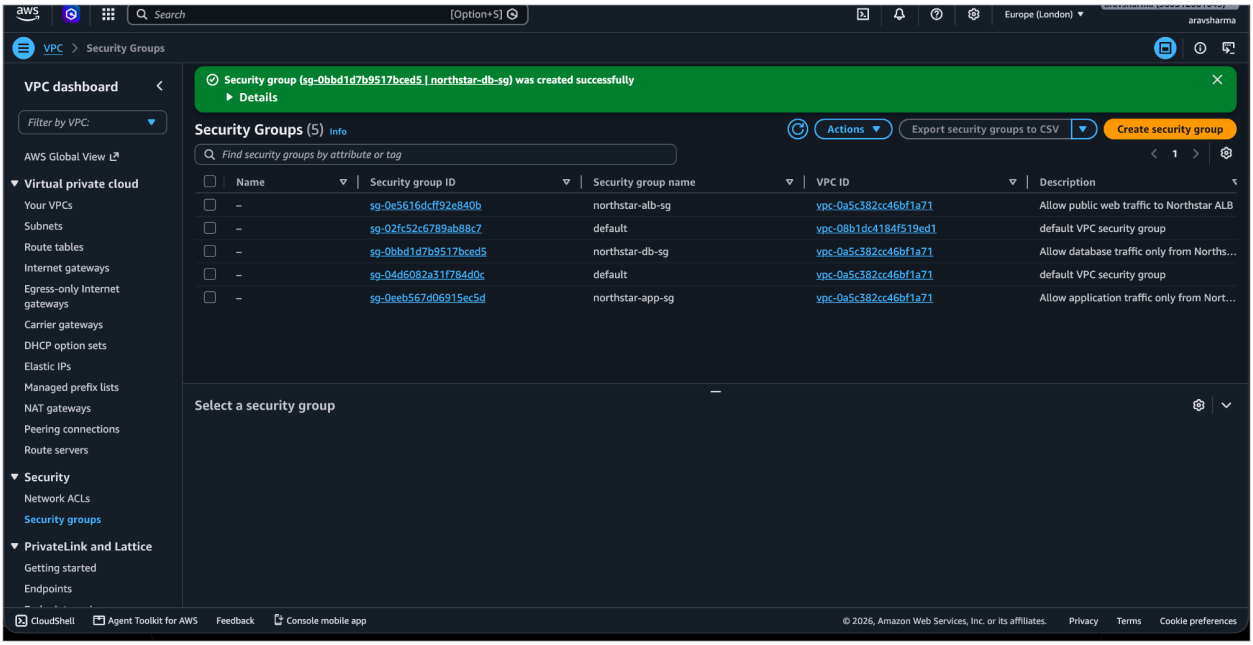


Figure 6. The three project security groups in northstar-vpc. The two “default” groups are created automatically with each VPC.

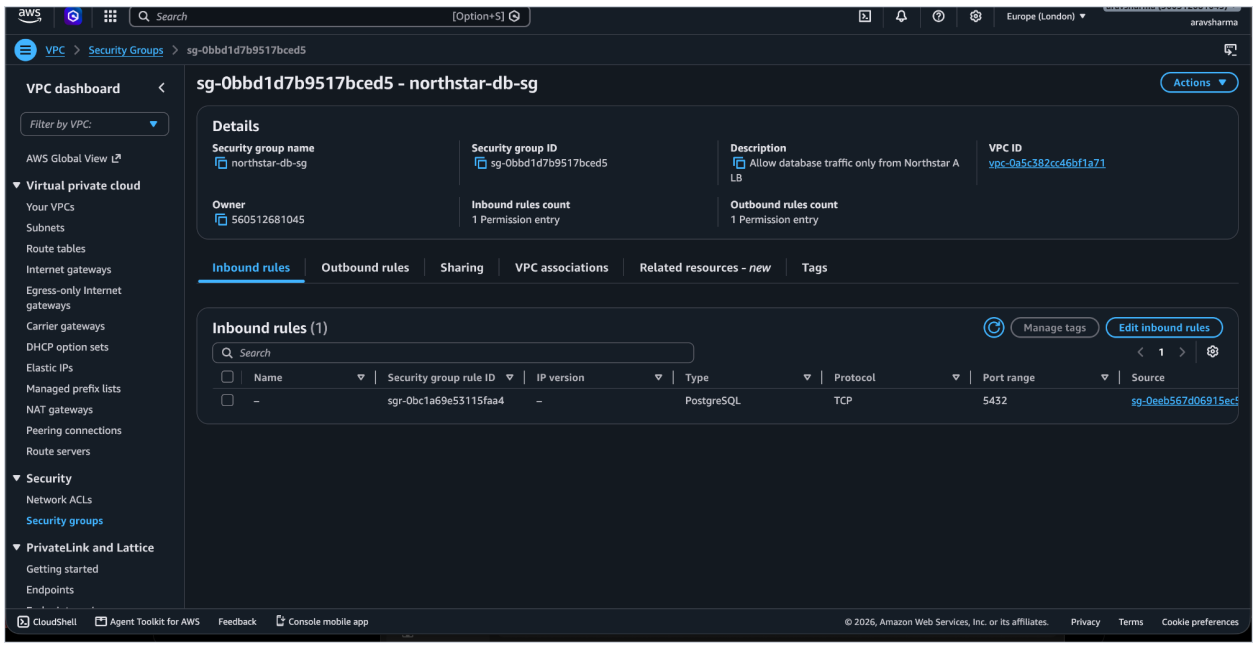


Figure 7. northstar-db-sg has a single inbound rule: PostgreSQL (TCP 5432) from the security group ID of northstar-app-sg.

4. Compute, admin access and deployment

The EC2 instance

The application runs on a t3.micro instance named northstar-app-1 in northstar-app-a, with the private IP `10.0.11.111` and no public IPv4 address. The instance requires IMDSv2 (the default for Amazon Linux 2023). IMDSv2 uses session tokens and blocks the simple GET requests that SSRF attacks rely on to steal instance credentials.

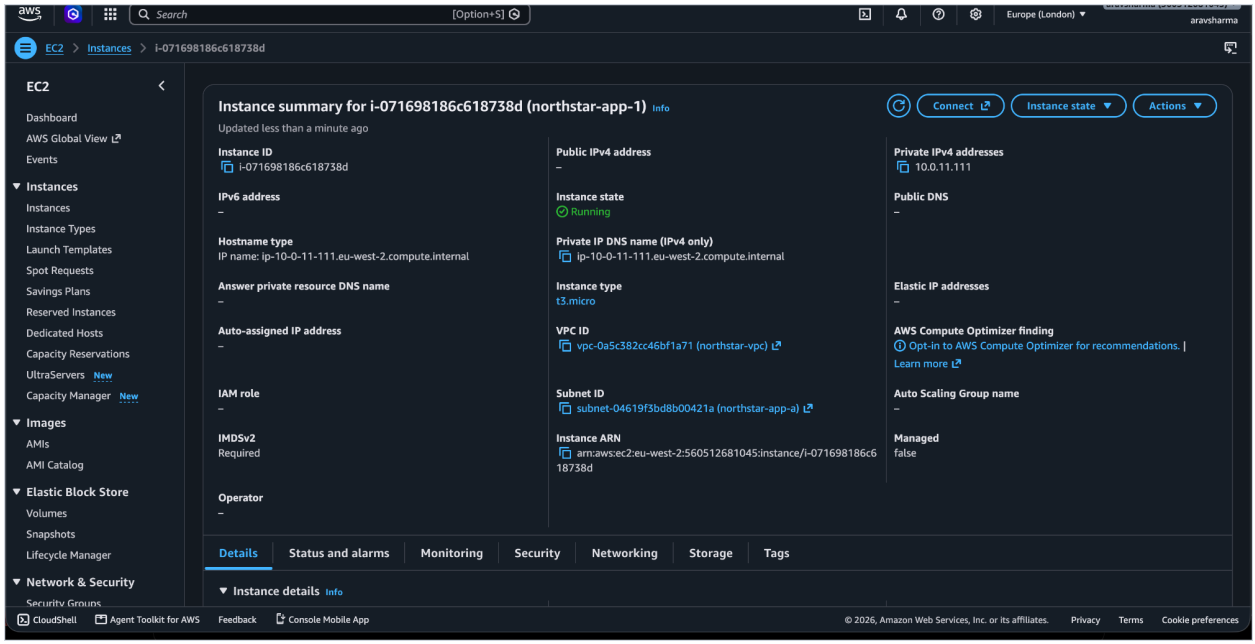


Figure 8. Instance summary: private IP 10.0.11.111, no public IPv4 address or public DNS, subnet northstar-app-a, IMDSv2 required.

Admin access with EC2 Instance Connect Endpoint

With no public IP, I couldn't SSH to the instance from my laptop directly. The usual workaround is a bastion host in a public subnet, but that is another server to patch and another open SSH port on the internet. Instead I created an EC2 Instance Connect Endpoint. It places a network interface inside the VPC; AWS checks my IAM permissions when I connect and then forwards the SSH session through that interface to the instance's private IP. The instance only needs to accept SSH from inside the VPC, never from the internet. The terminal screenshots later in this document were taken through this endpoint.

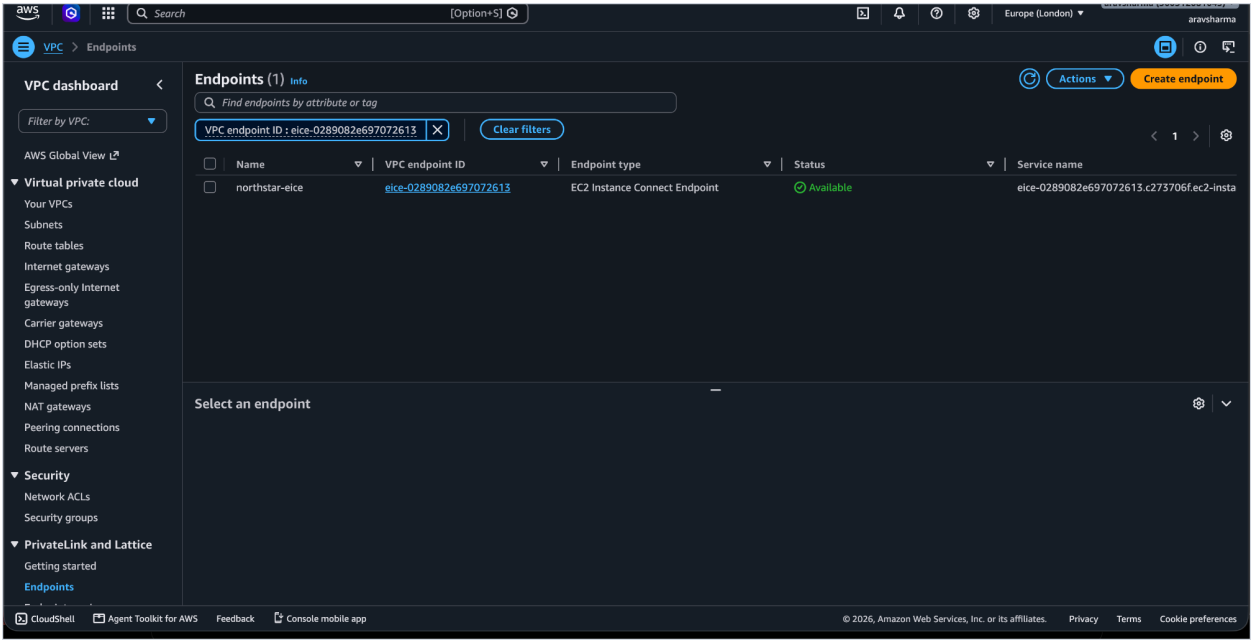


Figure 9. northstar-eice, the EC2 Instance Connect Endpoint, in the Available state.

Deploying the code through S3

The instance still needed to download the application code. Normally a private subnet would use a NAT gateway for outbound internet access, but a NAT gateway is charged by the hour and per GB, and it would give the server general internet access that it doesn't need. I uploaded the application files to a private S3 bucket and created an S3 Gateway VPC Endpoint attached to `northstar-app-rt`. Gateway endpoints for S3 have no extra charge. The endpoint adds a route for S3's address ranges to the route table, so requests from the instance to S3 stay on the AWS network, and the instance still has no route to anywhere else on the internet.

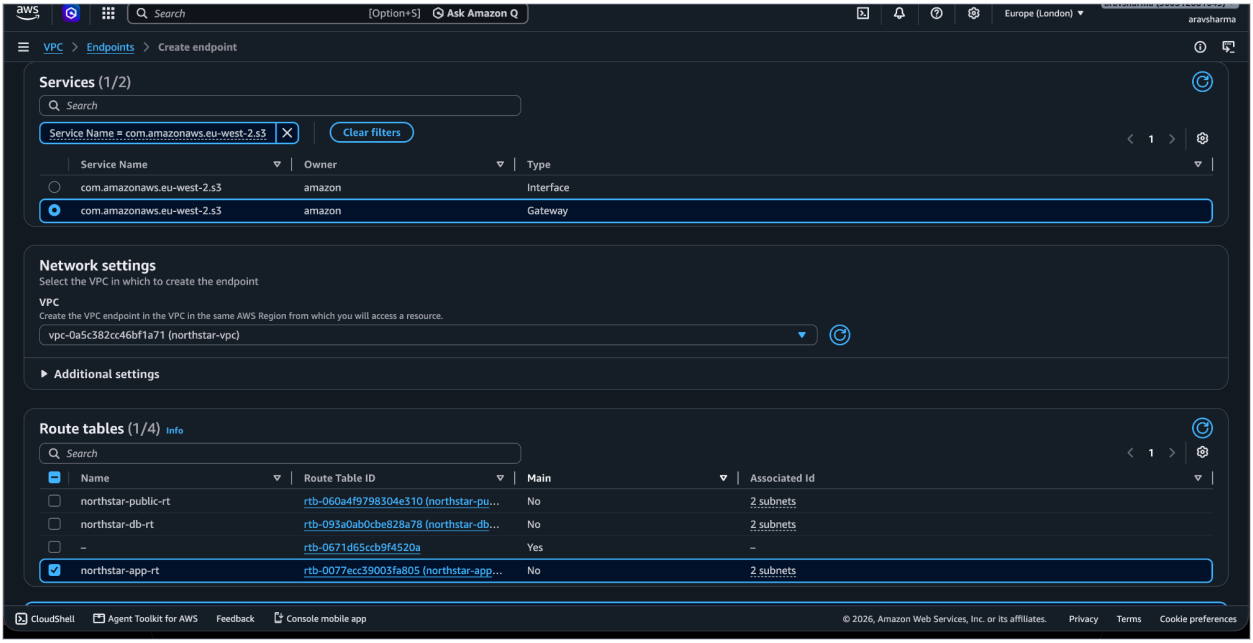


Figure 10. Creating the endpoint: the Gateway type of com.amazonaws.eu-west-2.s3, attached only to northstar-app-rt.

5. Database

RDS setup

The database is Amazon RDS for PostgreSQL (instance database-1) with 20 GiB of gp2 storage. It uses a DB subnet group, northstar-db-subnet-group, made of the two database subnets. RDS only places database instances in the subnets in that group. Because this is a Single-AZ deployment, the instance runs in eu-west-2a. With Multi-AZ turned on, RDS would put a standby copy in the eu-west-2b subnet and fail over to it automatically. I set "Publicly accessible" to No, so RDS never gives the database a public IP. The only security group attached is northstar-db-sg.

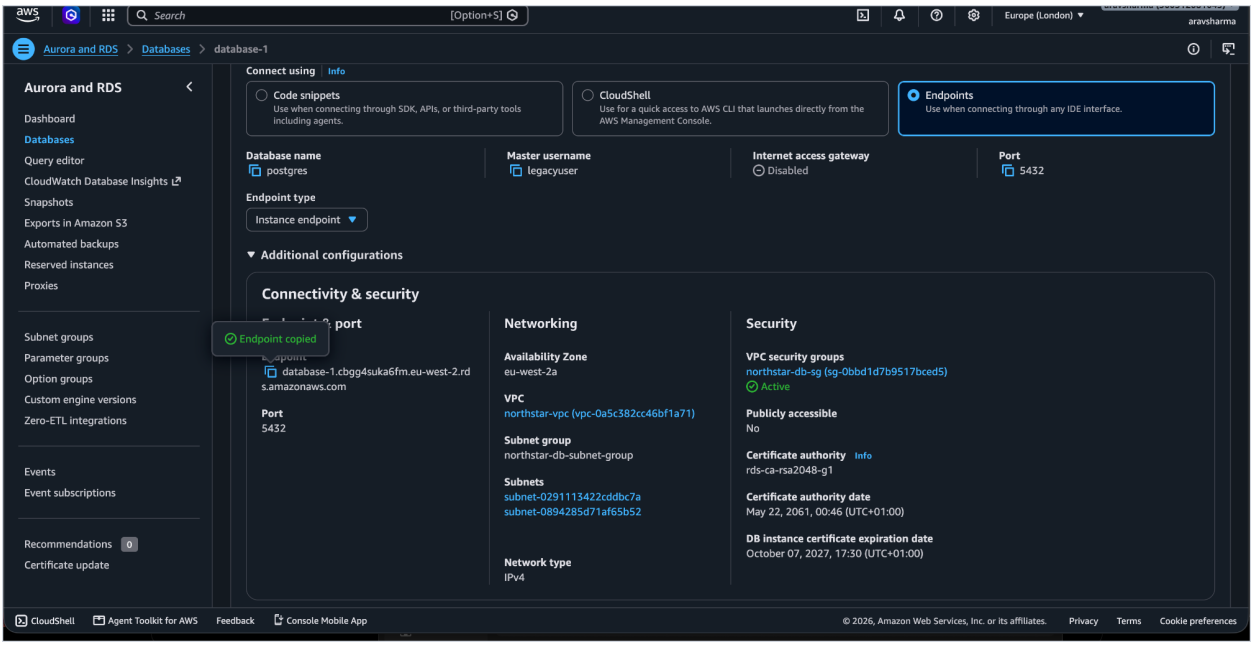


Figure 11. RDS connectivity settings: northstar-vpc, northstar-db-subnet-group, northstar-db-sg, publicly accessible No, eu-west-2a.

Connecting from the application tier

From the EC2 instance I tested the connection with a short Python script. It reads the host, port, database name, user and password from environment variables and connects with `sslmode="require"`, so the client refuses to connect unless the session is encrypted. The first test only checked that authentication worked.

```
    host=os.environ["DB_HOST"],
    port=os.environ["DB_PORT"],
    dbname=os.environ["DB_NAME"],
    user=os.environ["DB_USER"],
    password=os.environ["DB_PASSWORD"],
    sslmode="require"


rint("PostgreSQL authentication succeede
onn.close()
Y
ostgreSQL authentication succeeded
```

Figure 12. PostgreSQL authentication from the EC2 instance succeeded with SSL required.

I then created a `customers` table (id, name, a unique email and a created_at timestamp), inserted three sample rows and read them back. The insert uses `ON CONFLICT (email) DO NOTHING`, so the script can be run more than once without failing on duplicate emails.

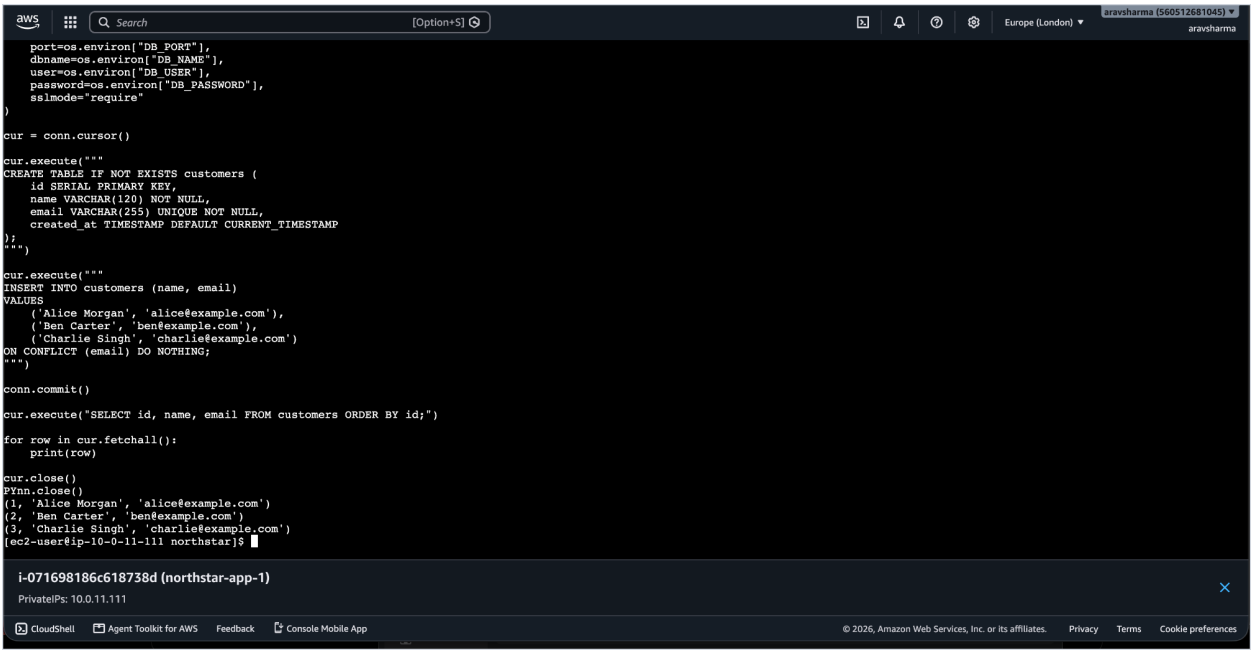


Figure 13. Creating the customers table, inserting three rows and reading them back from RDS.

How I troubleshot connection problems

When the database connection didn't work at first, I tested each layer separately instead of guessing:

1. DNS: does the RDS endpoint name resolve to a private address inside the VPC?
2. Network: can the instance open a TCP connection to port 5432? If not, the problem is a security group or a route, not the database.

3. Authentication: if TCP works, does the login succeed with the given user, password, database name and SSL setting?

Working through it in that order meant that a network problem was never mistaken for a wrong password, or the other way round. I used the same approach in the second failure test.

6. Application and end-to-end testing

Two health endpoints

The Flask app exposes two endpoints, and keeping them separate was a deliberate choice:

- `/health` returns `{"status":"ok"}` without touching the database. It only shows that the application process is running and able to answer requests. The ALB uses this endpoint for its health check.
- `/api/db-health` runs a simple query against PostgreSQL and returns `{"database":"reachable","result":1}`. It shows that the full path down to the database works.

If the ALB health check used the database endpoint, a database outage would make the ALB mark every app server as unhealthy and take the entire site offline, even though the servers themselves were fine. Keeping the load balancer check to the application process means a database problem shows up as failed database requests rather than a completely dead site. The second failure test shows this behaviour.

Testing each layer

I tested from the inside out. First, on the instance itself, I called both endpoints on `127.0.0.1:5000` to confirm that Gunicorn, Flask and the database connection all worked before involving the load balancer.

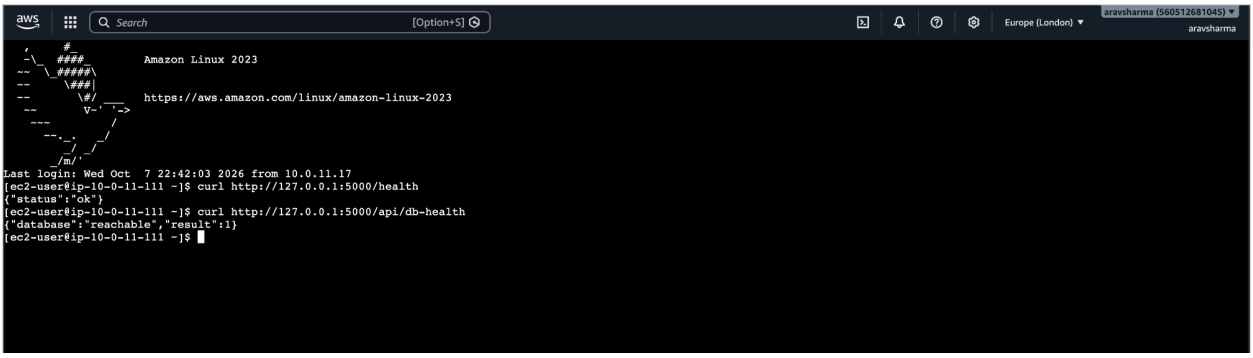


Figure 14. Local tests on the instance through EC2 Instance Connect: /health and /api/db-health both succeed.

Next I registered the instance in the target group northstar-app-tg (HTTP, port 5000) and checked that the ALB's health checks passed.

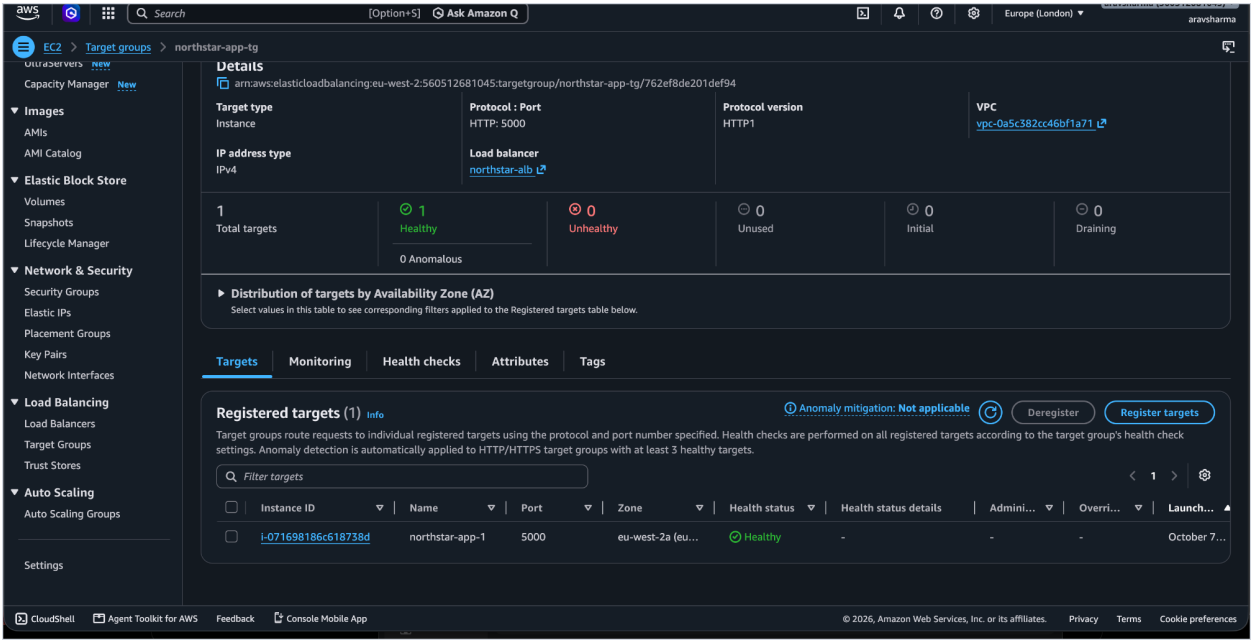


Figure 15. Target group northstar-app-tg: northstar-app-1 in eu-west-2a is Healthy on port 5000.

Finally I called both endpoints through the ALB's public DNS name from my own browser. This proves the full path: internet, ALB on port 80, EC2 on port 5000 and RDS on port 5432 over SSL.

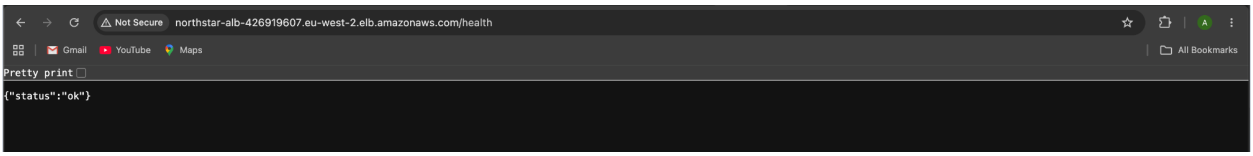


Figure 16. /health through the public ALB.

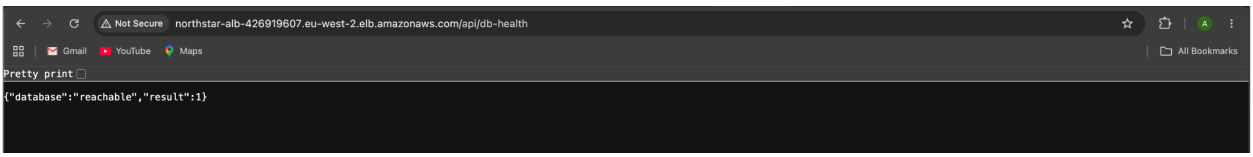


Figure 17. /api/db-health through the public ALB, confirming the request reached the database and back.

7. Failure test 1: stopping the application

Goal: confirm that the ALB detects a failed application server, see what a user experiences, and confirm that the service recovers once the process is back.

I stopped the Gunicorn service on northstar-app-1. Within a few health check intervals the target group marked the instance Unhealthy with the reason "Health checks failed".

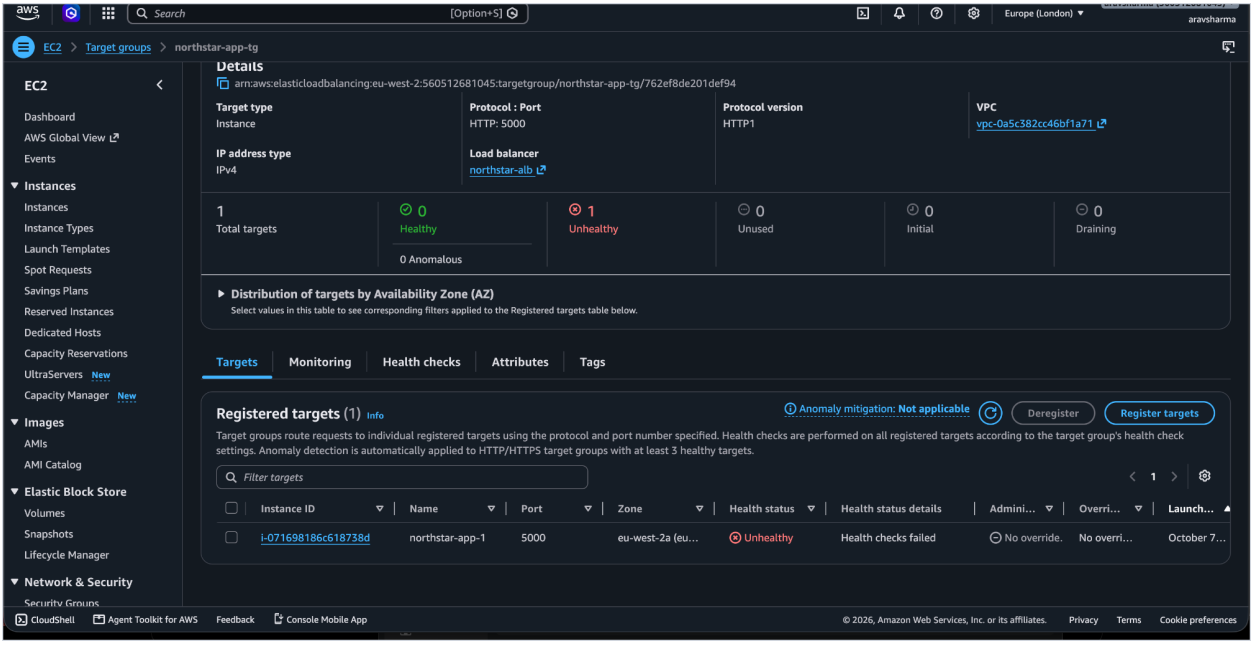


Figure 18. Target group after stopping Gunicorn: 0 healthy, 1 unhealthy, "Health checks failed".

Requests to the site returned 502 Bad Gateway. When every target in a group is unhealthy, the ALB "fails open" and keeps sending requests to them rather than dropping everything. Nothing was listening on port 5000, so the instance refused the connection, and the ALB reported that to the user as a 502.



Figure 19. What a user saw while Gunicorn was stopped: 502 Bad Gateway.

I started Gunicorn again. After the health checks passed, the target returned to Healthy and the site responded normally, with no changes needed on the ALB.

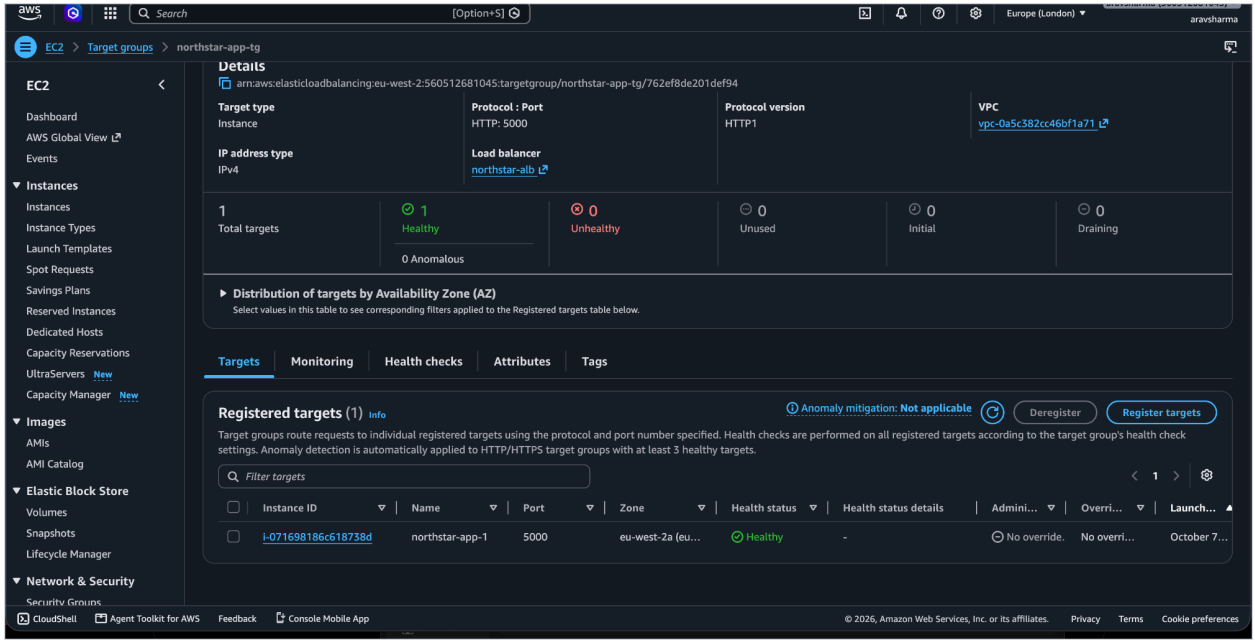


Figure 20. After restarting Gunicorn, the target is Healthy again.

Stage	What happened
Fault	Stopped Gunicorn on northstar-app-1
Detection	ALB health checks on /health failed and the target was marked Unhealthy
User impact	Every request returned 502 Bad Gateway
Recovery	Restarted Gunicorn; the target passed health checks and traffic returned to normal

What this showed: with only one server, the health check can detect the failure but has nowhere else to send traffic, so the whole site goes down. In production I would run at least two instances in an Auto Scaling Group across both Availability Zones. The ALB would then route around the failed instance, and the Auto Scaling Group would replace it.

8. Failure test 2: blocking database access

Goal: break the network path between the application and the database, and check whether the health endpoints make it obvious which layer has failed.

I deleted the inbound PostgreSQL rule from northstar-db-sg, so the database stopped accepting connections from the app tier. The EC2 instance, Gunicorn and the ALB were left untouched.

`/health` kept returning OK through the ALB. Because the ALB health check uses that endpoint, the instance stayed in service. `/api/db-health` failed with a 502. This combination pointed straight at the connection between the app and the database: if the ALB or the application had been broken, `/health` would have failed too.

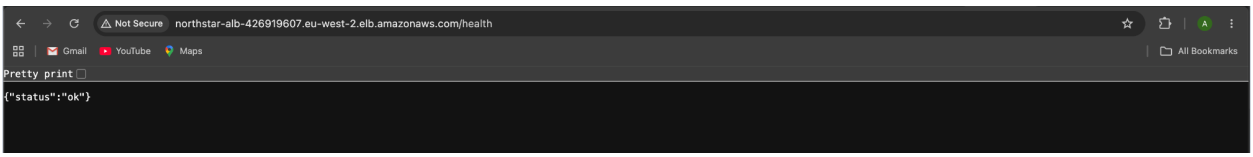


Figure 21. /health still returns OK while database access is blocked, so the ALB and the application are fine.



Figure 22. /api/db-health returns 502 while the database rule is removed.

Why a 502 and not a clear database error? When a security group blocks traffic, it drops the packets silently rather than rejecting them. The application's connection attempt therefore hangs instead of failing quickly. Most likely the Gunicorn worker hit its timeout and was killed before it could send a response, and the ALB reported the dropped connection as a 502. A better design would set a short `connect_timeout` on the database connection and return a 503 with a clear message, so the failure is fast and easy to diagnose.

I added the inbound rule back (PostgreSQL 5432 from northstar-app-sg) and `/api/db-health` immediately returned "reachable" again. No changes were needed to the instance, the application or the ALB.

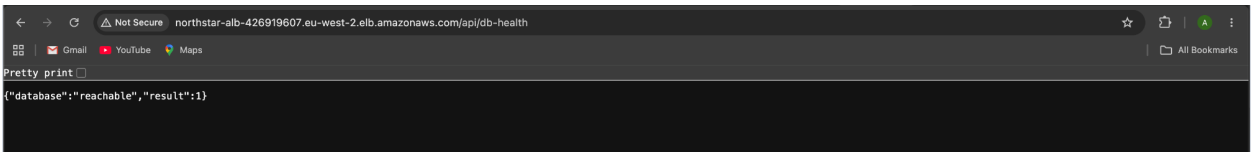


Figure 23. /api/db-health working again after the security group rule was restored.

9. Backup and recovery

Once the database was set up, I took a manual snapshot of the RDS instance. It completed on 7 October 2026 and shows as Available.

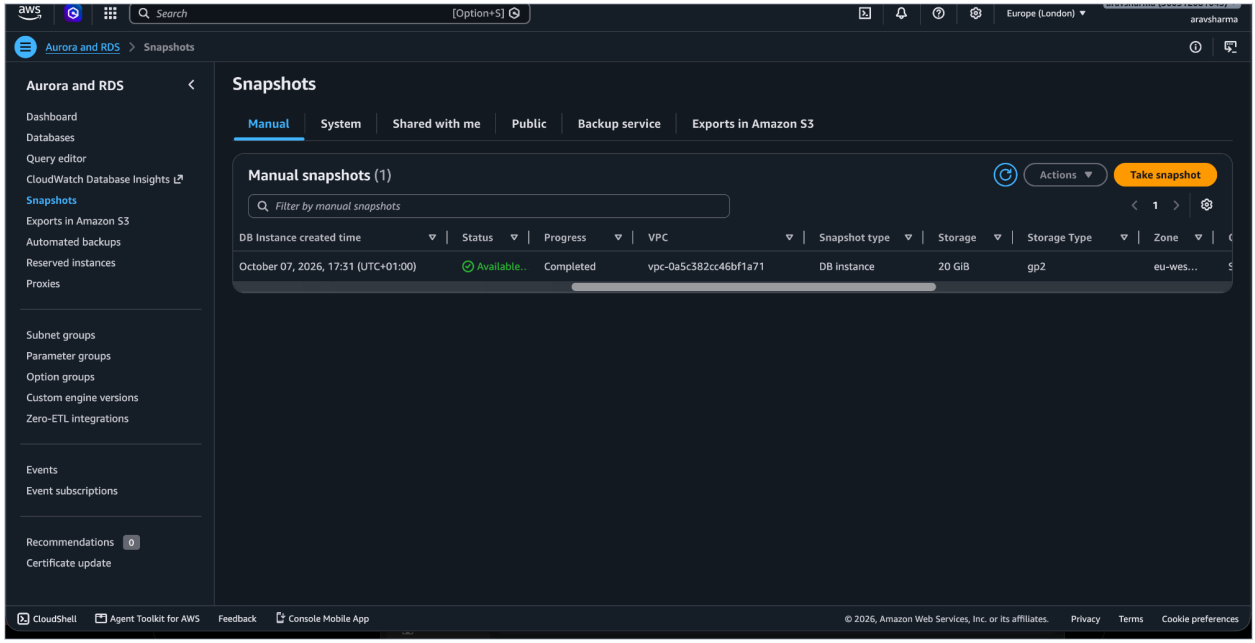


Figure 24. Manual RDS snapshot: Available, 20 GiB, in northstar-vpc.

I did not restore from it, so I haven't measured a recovery time objective (RTO, how long recovery takes) or a recovery point objective (RPO, how much data could be lost). A restore would create a new RDS instance with a new endpoint, so the test would be: restore the snapshot, point the application's `DB_HOST` at the new endpoint, run `/api/db-health` and the customers query, and time the whole process. With only manual snapshots, the RPO is however long it has been since the last snapshot. With RDS automated backups turned on, point-in-time restore can usually bring the database back to within about five minutes of the failure.

10. Changes for production

Change	Reason
Auto Scaling Group with at least two instances across both AZs	Removes the single point of failure shown in failure test 1; failed instances are replaced automatically
Multi-AZ RDS	A standby in eu-west-2b with automatic failover if the primary or its AZ fails
HTTPS listener with an ACM certificate, and redirect HTTP to HTTPS	Encrypts traffic between users and the ALB; the lab used plain HTTP
AWS Secrets Manager for database credentials	The lab passed the password through environment variables; Secrets Manager stores it encrypted and can rotate it
Database connection timeout and a 503 response	Makes database failures fast and clearly reported instead of a 502 after a timeout (failure test 2)
CloudWatch alarms and centralised logs	Alert on unhealthy hosts and 5xx errors instead of finding problems by refreshing a browser
Automated RDS backups and a tested restore	Gives a real, measured RTO and RPO instead of an untested snapshot
Infrastructure as Code (Terraform or CloudFormation)	The whole environment can be rebuilt, reviewed and versioned instead of clicked together by hand
CI/CD pipeline with smoke tests	Deploy new versions automatically and only send traffic to an instance after /health and /api/db-health pass

11. What I learned

Building every part by hand made it clear how the layers depend on each other. A subnet is only public because of its route table, and a database is only private because of the subnet group, the security group and the publicly accessible setting all agreeing. The two failure tests were the most useful part of the project. The first showed that a health check is only worth as much as the redundancy behind it. The second showed that splitting health endpoints by dependency turns a vague “the site is broken” into a specific answer about which connection failed. Next, I want to rebuild the same architecture in Terraform with an Auto Scaling Group and Multi-AZ RDS, and actually measure a restore.